

Lab 4: LLM Practice

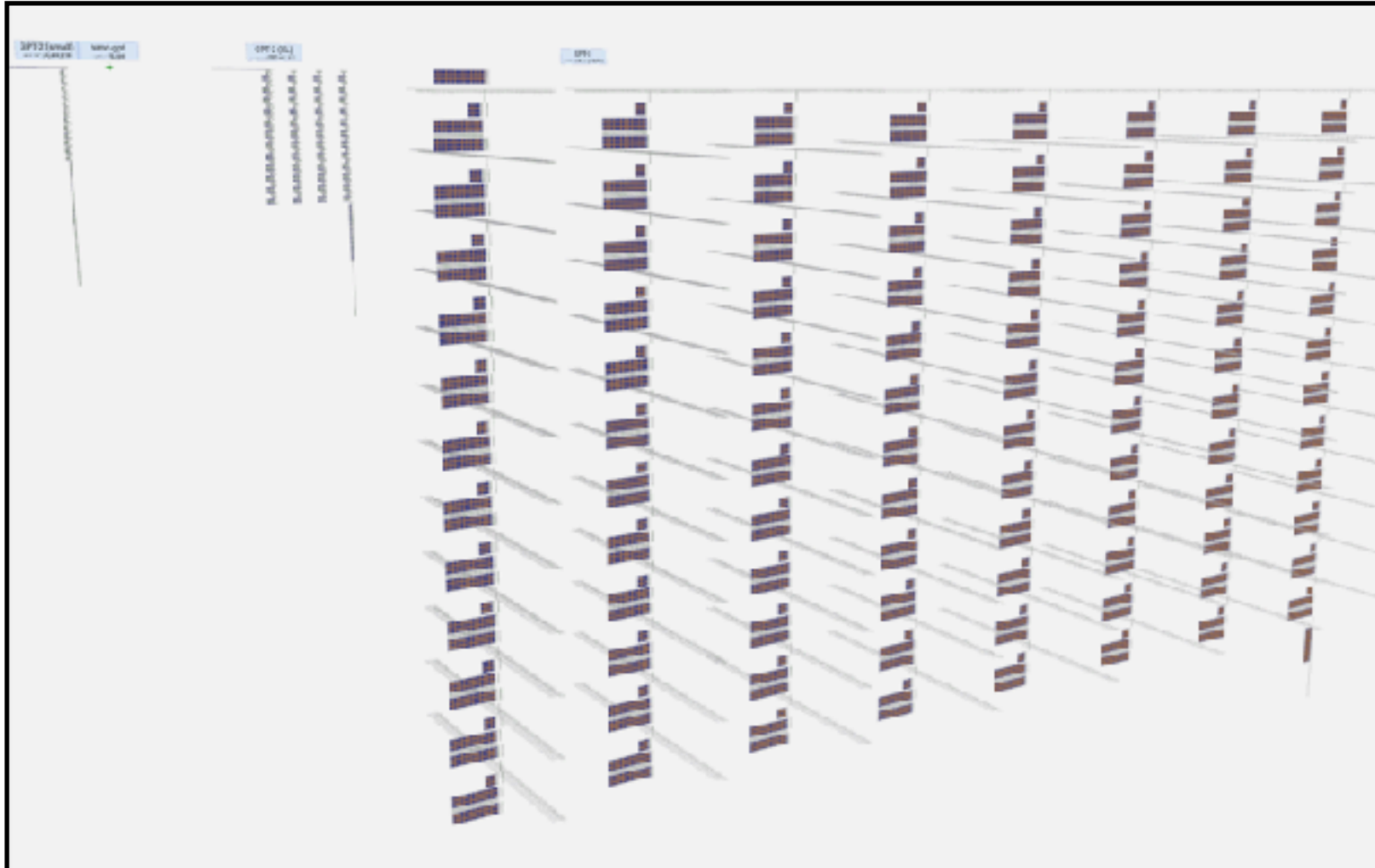
<삼성 AI 전문가 교육과정> 실습
서울대학교 바이오지능 연구실 (장병탁 교수)
정성준, 최우석
2025.05.29

Biointelligence Laboratory
Dept. of Computer Science and Engineering
Seoul National University



Visualization: How do LLMs work?

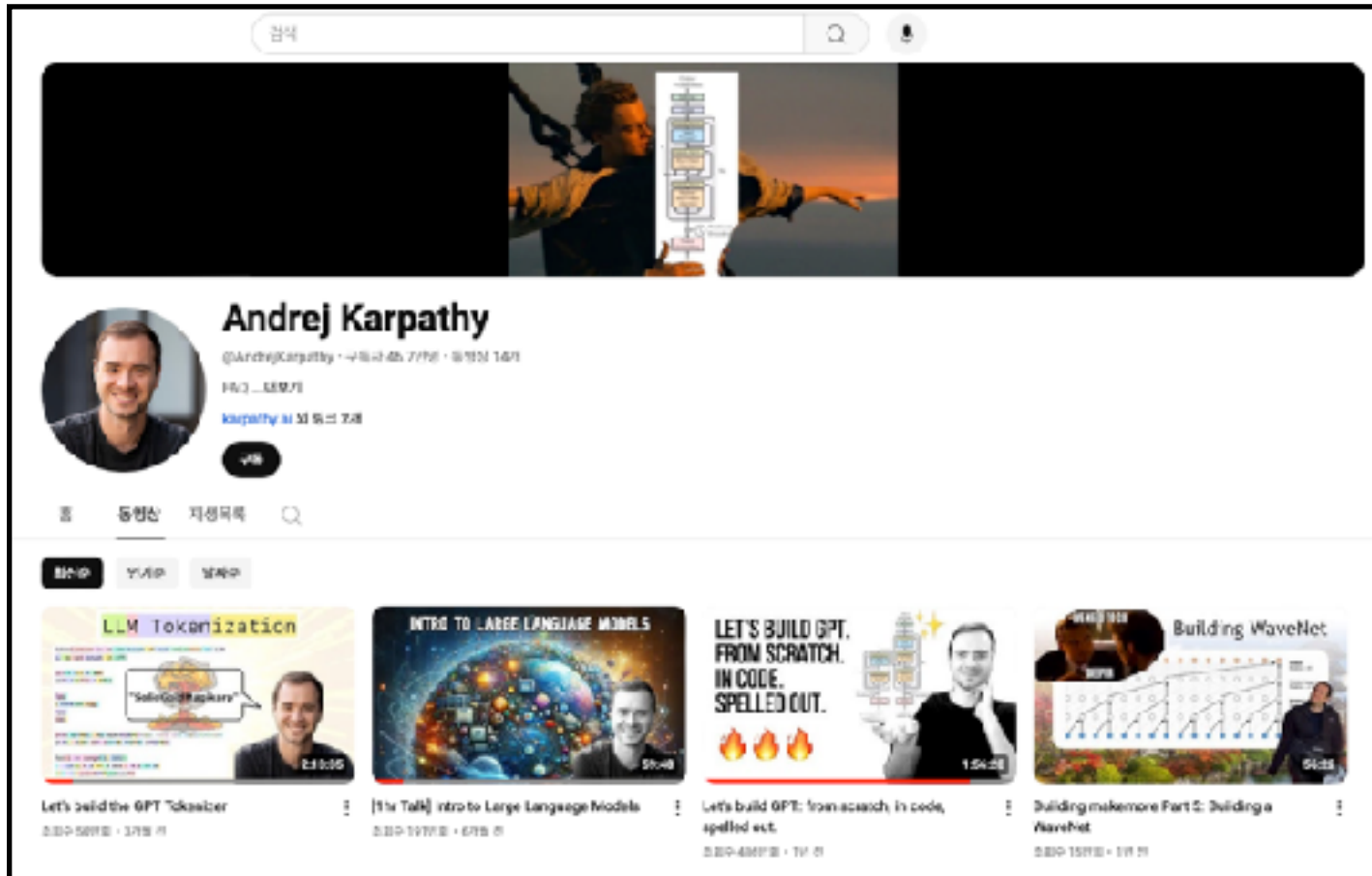
<https://bbycroft.net/llm>



Andrej Karpathy's Lecture

<https://www.youtube.com/@AndrejKarpathy>

<https://karpathy.ai/zero-to-hero.html>



The image shows the YouTube channel page for Andrej Karpathy. At the top is a banner image of him pointing at a diagram. Below the banner is his profile picture and name, "Andrej Karpathy", with his handle "@AndrejKarpathy" and subscriber count "147K". Underneath is a row of tabs: "홈" (Home), "동영상" (Videos), "자세히 보기" (See more), and "검색" (Search). Below the tabs are four video thumbnails:

- LLM Tokenization**: "Let's build the GPT Tokenizer" (0:00-5:01) • 171K views
- INTRO TO LARGE LANGUAGE MODELS**: "[1h Talk] intro to Large Language Models" (0:00-1:07) • 675K views
- Let's build GPT, FROM SCRATCH. IN CODE. SPELLED OUT.**: "Let's build GPT: from scratch, in code, spelled out." (0:00-4:00) • 1.5M views
- Building WaveNet**: "Building makemore Part 2: Building a WaveNet" (0:00-1:07) • 171K views



The image shows the "Neural Networks: Zero to Hero" course page. At the top is the title "Neural Networks: Zero to Hero". Below it is a description: "A course by Andrej Karpathy on building neural networks, from scratch, in code. We start with the basics of backpropagation and build up to modern deep neural networks, like GPT. In my opinion, language models are an excellent place to learn deep learning, even if your intention is to eventually go to other areas like computer vision because most of what you learn will be immediately transferable. This is why we dive into and focus on language models. Prerequisite: solid programming (Python), intro-level math (e.g. derivative, gradient)." Below the description is a button "Learn more" and a link "Neural Networks: Zero to Hero".

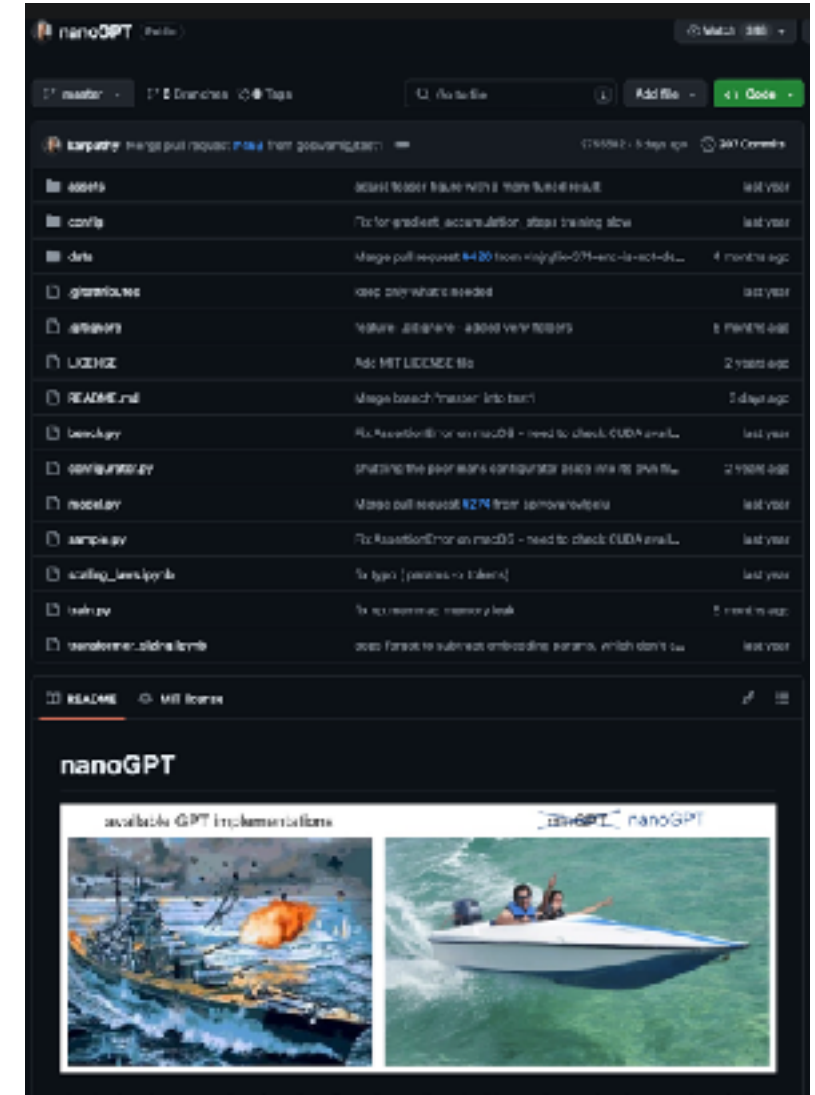
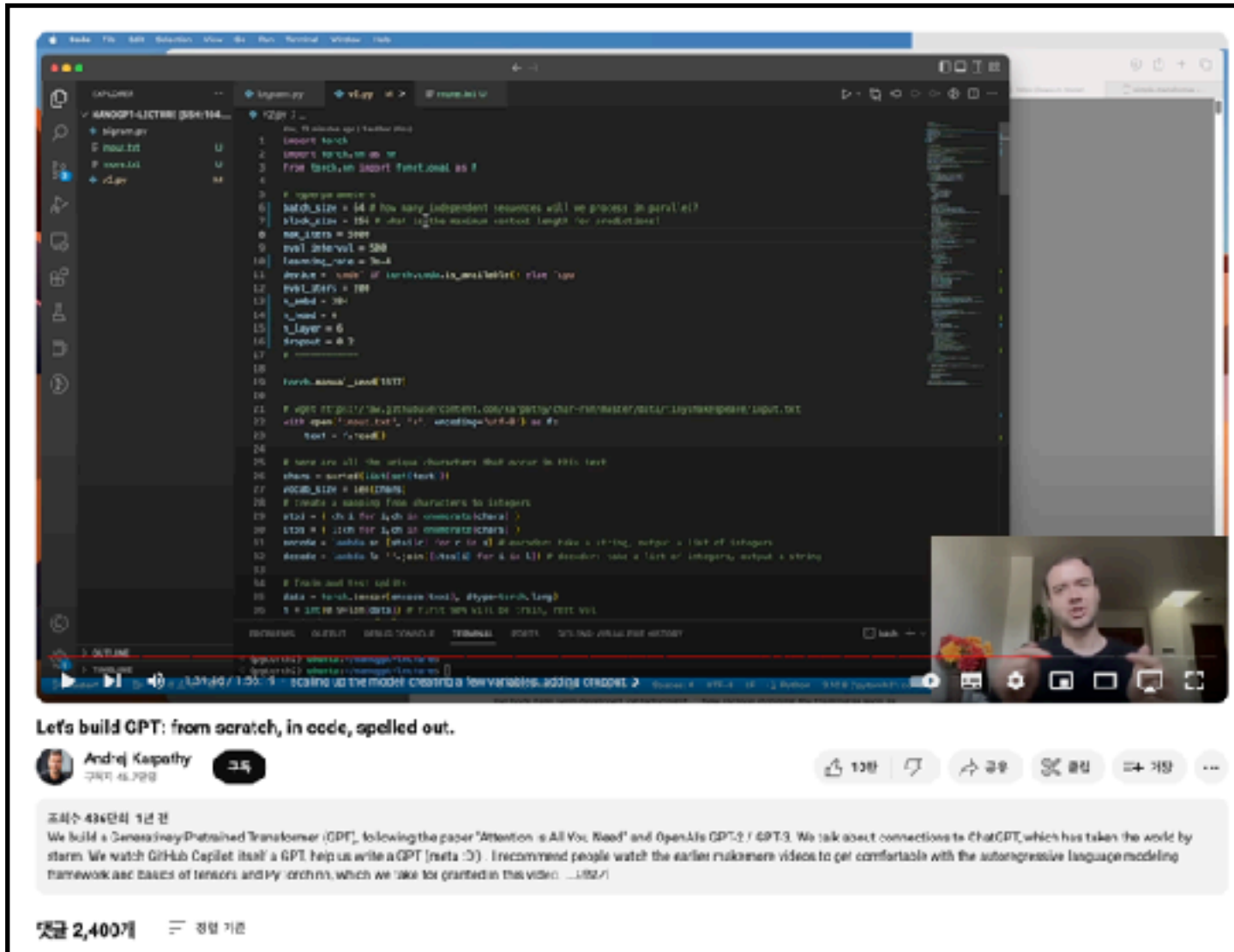
Syllabus

- 2:25m The spelled-out intro to neural networks and backpropagation, building makemore**
This is the most step-by-step spelled-out explanation of backpropagation and training of neural networks. It only assumes basic knowledge of Python and a vague recollection of calculus from high school.
- 1:07m The spelled-out intro to language modeling, building makemore**
We implement a simple character-level language model, which we will further complicate in follow-up videos into a modern transformer language model, like GPT. In this video, the focus is on (1) introducing tokenization and its subtleties and use in efficiently evaluating neural networks and (2) the overall framework of language modeling that includes model training, sampling, and the evaluation of a loss (e.g. the negative log likelihood for classification).
- 1:15m Building makemore Part 2: MLP**
We implement a multilayer perceptron (MLP) character-level language model. In this video we also introduce many basics of machine learning (e.g. model training, learning rate tuning, hyperparameters, evaluation, L1/L2/ReLU/softmax, weights/initialization, etc.).
- 1:55m Building makemore Part 3: Activations & Gradients, Batch Norm**
We dive into some of the intricacies of MLPs with multiple layers and scrutinize the statistics of the forward pass activations, backward pass gradients, and some of the pitfalls when they are improperly scaled. We also look at the typical diagnostic tools and visualizations you'll want to use to understand the health of your deep network. We learn why training deep neural nets can be fragile and introduce the first modern innovation that cures being so much easier: Patch-Matrix-vector Product connections and the Adam optimizer remain notable topics for later videos.
- 1:55m Building makemore Part 4: Becoming a GPT-like Ninja**
We build the GPT-like Transformer-style model, from the very beginning, and we'll see how it works.

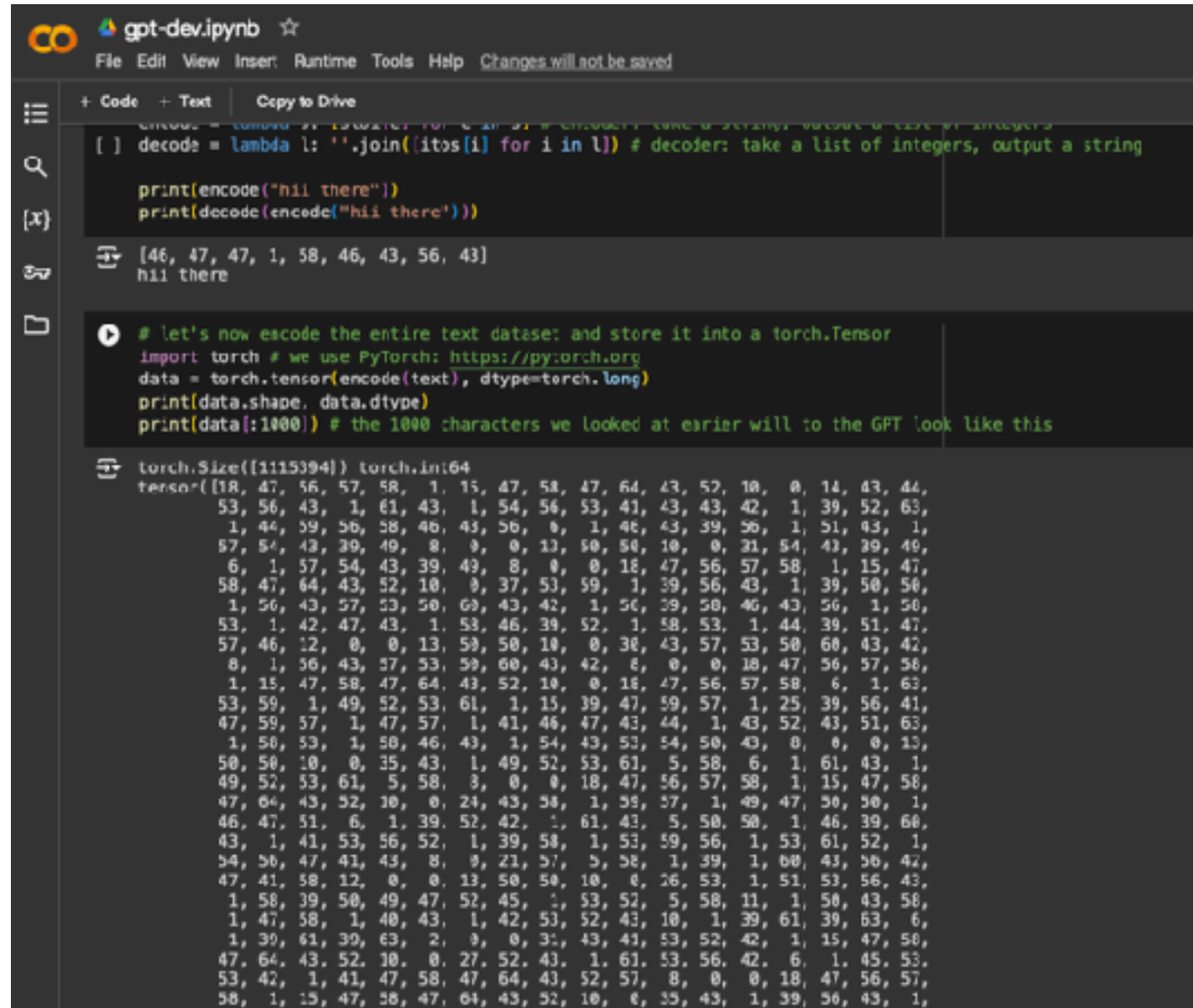
Andrej Karpathy's GPT Tutorial

<https://www.youtube.com/@AndrejKarpathy>

<https://github.com/karpathy/nanoGPT>



Andrej Karpathy's GPT Tutorial



```
gpt-dev.ipynb
File Edit View Insert Runtime Tools Help Changes will not be saved

+ Code + Text Copy to Drive

[ ] encode = lambda s: [tokenizer.get_index_with_max_prob(tokenizer.encode(s)) for token in tokenizer.tokenize(s)]
[ ] decode = lambda l: ''.join([tokens[i] for i in l]) # decoder: take a list of integers, output a string

print(encode("hi there"))
print(decode(encode("hi there")))
```

[46, 47, 47, 1, 58, 46, 43, 56, 43]
hi there

```
# let's now encode the entire text dataset and store it into a torch.Tensor
import torch # we use PyTorch: https://pytorch.org
data = torch.tensor(encode(text), dtype=torch.long)
print(data.shape, data.dtype)
print(data[:1000]) # the 1000 characters we looked at earlier will to the GPT look like this
```

torch.Size([1115394]) torch.int64
tensor([18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 57, 10, 0, 14, 43, 44, 53, 56, 43, 1, 61, 43, 1, 54, 56, 53, 41, 43, 43, 42, 1, 39, 52, 63, 1, 44, 59, 56, 58, 46, 43, 56, 0, 1, 46, 43, 39, 56, 1, 51, 43, 1, 57, 54, 43, 39, 49, 8, 0, 0, 13, 50, 50, 10, 0, 31, 54, 43, 39, 49, 6, 1, 57, 54, 43, 39, 49, 8, 0, 0, 18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 37, 53, 59, 1, 39, 56, 43, 1, 39, 50, 50, 1, 56, 43, 57, 53, 50, 60, 43, 42, 1, 56, 39, 50, 46, 43, 56, 1, 50, 53, 1, 42, 47, 43, 1, 53, 46, 39, 52, 1, 58, 53, 1, 44, 39, 51, 47, 57, 46, 12, 0, 0, 13, 50, 50, 10, 0, 30, 43, 57, 53, 50, 60, 43, 42, 0, 1, 56, 43, 57, 53, 50, 60, 43, 42, 0, 0, 0, 18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 18, 47, 56, 57, 58, 6, 1, 63, 53, 59, 1, 49, 52, 53, 61, 1, 15, 39, 47, 59, 57, 1, 25, 39, 56, 41, 47, 59, 57, 1, 47, 57, 1, 41, 46, 47, 43, 44, 1, 43, 52, 43, 51, 63, 1, 50, 53, 1, 50, 46, 43, 1, 54, 43, 53, 54, 50, 43, 0, 0, 0, 13, 50, 50, 10, 0, 35, 43, 1, 49, 52, 53, 61, 5, 58, 6, 1, 61, 43, 1, 49, 52, 53, 61, 5, 58, 3, 0, 0, 18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 24, 43, 50, 1, 50, 57, 1, 49, 47, 50, 50, 1, 46, 47, 51, 6, 1, 39, 52, 42, 1, 61, 43, 5, 50, 50, 1, 46, 39, 60, 43, 1, 41, 53, 56, 52, 1, 39, 58, 1, 53, 59, 56, 1, 53, 61, 52, 1, 54, 56, 47, 41, 43, 8, 0, 21, 57, 5, 58, 1, 39, 1, 60, 43, 50, 42, 47, 41, 58, 12, 0, 0, 13, 50, 50, 10, 0, 26, 53, 1, 51, 53, 56, 43, 1, 58, 39, 50, 49, 47, 52, 45, 1, 53, 52, 5, 58, 11, 1, 50, 43, 58, 1, 47, 58, 1, 40, 43, 1, 42, 53, 52, 43, 10, 1, 39, 61, 39, 63, 6, 1, 39, 61, 39, 63, 2, 0, 0, 31, 43, 41, 53, 52, 42, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 27, 52, 43, 1, 61, 53, 56, 42, 6, 1, 45, 53, 53, 42, 1, 41, 47, 58, 47, 64, 43, 52, 57, 8, 0, 0, 18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 35, 43, 1, 39, 56, 43, 1,

GPT fine tuning

<https://github.com/karpathy/nanoGPT> (300 lines)

Replication: 8X A100 40GB이상 필요

finetuning

Finetuning is no different than training, we just make sure to initialize from a pretrained model and train with a smaller learning rate. For an example of how to finetune a GPT on new text go to `data/shakespeare` and run `prepare.py` to download the tiny shakespeare dataset and render it into a `train.bin` and `val.bin`, using the OpenAI BPE tokenizer from GPT-2. Unlike OpenWebText this will run in seconds. Finetuning can take very little time, e.g. on a single GPU just a few minutes. Run an example finetuning like:

```
python train.py config/finetune_shakespeare.py
```

This will load the config parameter overrides in `config/finetune_shakespeare.py` (I didn't tune them much though). Basically we initialize from a GPT2 checkpoint with `init_from` and train as normal, except shorter and with a small learning rate. If you're running out of memory try decreasing the model size (they are `{'gpt2', 'gpt2-medium', 'gpt2-large', 'gpt2-xl'}`) or possibly decreasing the `block_size` (context length). The best checkpoint (lowest validation loss) will be in the `out_dir` directory, e.g. in `out-shakespeare` by default, per the config file. You can then run the code in `sample.py --out_dir=out-shakespeare`:

```
THEODORE:
Thou shalt sell me to the highest bidder: if I die,
I sell thee to the first; if I go mad,
I sell thee to the second; if I
lie, I sell thee to the third; if I slay,
I sell thee to the fourth: so buy or sell,
I tell thee again, thou shalt not sell my
possession.
```

```
JULIET:
And if thou steal, thou shalt not sell thyself.
```

```
THEANON:
I do not steal; I sell the stolen goods.
```

```
THEODORE:
Thou know'st not what thou sell'st; thou, a woman,
Thou art ever a victim, a thing of no worth:
Thou hast no right, no right, but to be sold.
```

Whoa there, GPT, entering some dark place over there. I didn't really tune the hyperparameters in the config too much, fee free to try!

```
running_ema = 1.0
while True:

    # determine and set the learning rate for this iteration
    lr = get_lr(iter_num) if decay_lr else learning_rate
    for param_group in optimizer.param_groups:
        param_group['lr'] = lr

    # evaluate the loss on train/val sets and with checkpoints
    if iter_num % eval_interval == 0 and master_process:
        losses = estimate_loss()
        print(f"step {iter_num}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")
        if wandb_log:
            wandb.log({
                "iter": iter_num,
                "train/loss": losses["train"],
                "val/loss": losses["val"],
                "lr": lr,
                "eta": running_ema*100, # convert to percentage
            })
        if losses['val'] < best_val_loss or always_save_checkpoint:
            best_val_loss = losses['val']
            if iter_num % 1000 == 0:
                checkpoint = {
                    'model': raw_model.state_dict(),
                    'optimizer': optimizer.state_dict(),
                    'model_args': model_args,
                    'iter_num': iter_num,
                    'best_val_loss': best_val_loss,
                    'config': config,
                }
                print(f"saving checkpoint to {out_dir}")
                torch.save(checkpoint, os.path.join(out_dir, f'ckpt.pt'))
        if iter_num == 0 and eval_only:
            break

    # forward backward updates, with optional gradient accumulation to simulate larger batch size
    # and using the GradScaler if data type is float16
    for micro_step in range(gradient_accumulation_steps):
        if ddp:
            # in DDP training we only need to sync gradients at the last micro step.
            # the official way to do this is with sync_context_manager, but
            # it really messes up the code and forces us to repeat some
            # looking at the source of that context manager, it just toggles this variable
            model.require_backward_grad_sync = (micro_step == gradient_accumulation_steps - 1)
        with torch.no_grad():
            logits, loss = model(x, y)
            loss = loss / gradient_accumulation_steps # scale the loss to account for gradient accumulation
        # immediately async prefetch next batch while model is doing the forward pass on the GPU
        x, y = get_batch('train')
        # backward pass, with gradient scaling if training in fp16
        scaler.scale(loss).backward()
    # clip the gradient
    if grad_clip is not None:
        scaler.unscale_._for_master_grads()
        torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
```

Andrej Karpathy's GPT Tokenizer Tutorial

<https://www.youtube.com/@AndrejKarpathy>

<https://tiktokenizer.vercel.app/>

The screenshot shows a Telegram chat window titled "Tiktokerizer". At the top, there is a system message: "System: You are a helpful assistant." Below this, a user message "Hello" is visible. The bot's response is a list of numbers: 4421, 2560, 102, 566, 293, 5006, 320, 2669, 26710, 2016, 328, 492, 19041, 82, 13, 3701, 620, 24809, 480, 1277, 366, 12817, 559, 229, 48806, 314, 228, 89524, 190, 18887, 290, 609, 220, 45800, 10, 311, 750, 32556, 16, 299, 189338, 558, 48, 679, 448, 82711, 568, 71126, 688, 21289, 28, 345, 18452, 7663, 4861, 38897, 4881, 30771, 2502, 11, 124800, 7710, 12127, 10440, 81108, 1748, 18665, 51680, 17567, 65163, 1960, 128572, 5544, 17651, 142816, 27931, 13, 187867, 16618, 2748, 44947, 125416, 122915, 131740, 2180, 26156, 45431, 17915, 8122, 37436, 364, 1931, 575, 564, 9112, 7, 16, 11, 120, 7063, 1813, 271, 138, 876, 1661, 220, 18, 661, 220, 16, 326, 576, 1081, 220, 20, 502, 220, 16, 734, 169, 2128, 648, 198746, 84716, 1884, 271, 9127, 876, 1861, 220, 18, 951, 220, 15, 734, 109, 2120, 566, 198746, 1896, 271, 9449, 576, 1891, 220, 20, 502, 220, 16, 734, 169, 2128, 648, 198746, 84716, 1884, 271, 1281, 734, 169, 2121, 2049, 1.

Andrej Karpathy's GPT Tokenizer Tutorial

```
Lab4_tokenization_karpathy.ipynb
File Edit View Insert Runtime Tools Help

+ Code + Text Copy to Drive

[ ] 180,
[ ] 180,
[ ] 111,
[ ] 32,
[ ] 180,
[ ] 110,
[ ] 32,
[ ] 75,
[ ] 111,
[ ] 110,
[ ] 101,
[ ] 90,
[ ] 110,
[ ] 32,
[ ] 41]

[ ] # text from https://www.reddit.com/blog/programmers-Intro-to-unicode/
[ ] text = "lll llllll 00000000 00000000 ! The very name strikes fear and awe into the hearts of programmers"
[ ] tokens = text.encode("utf-8") # raw bytes
[ ] tokens = list(map(int, tokens)) # convert to a list of integers in range 0..255 for convenience
[ ] print('---')
[ ] print(text)
[ ] print("length:", len(text))
[ ] print('---')
[ ] print(tokens)
[ ] print("length:", len(tokens))

[ ] ---
[ ] Unicode: 00000000 00000000 ! The very name strikes fear and awe into the hearts of programmers worl
[ ] length: 133
[ ] ---
[ ] [250, 180, 180, 230, 160, 140, 230, 130, 130, 230, 180, 140, 250, 180, 130, 230, 160, 130, 32, 20
[ ] length: 616

[ ] def get_stats(ids):
[ ]     counts = {}
[ ]     for pair in zip(ids, ids[1:]): # Pythonic way to iterate consecutive elements
[ ]         counts[pair] = counts.get(pair, 0) + 1
[ ]     return counts

[ ] stats = get_stats(tokens)
[ ] # print(stats)
[ ] # print(sorted(kv for k,v in stats.items(), reverse=True))

[ ] top_pair = max(stats, key=stats.get)
[ ] top_pair

[ ] (181, 32)

[ ] def merge(ids, pair, id*):
[ ]     # in the list of ints (ids), replace all consecutive occurrences of pair with the new token id*
[ ]     newids = []
[ ]     i = 0
```

Commercial Open-source LLM (1)

■ Gemma (2024.02)

- Google/US
- Gemini 1.0 기반 경량 구조
- 2B, 7B, 14B, 27B
- 64K context length

■ Phi-3 (2024.03)

- Microsoft/US
- MIT License (무료)
- 3.8B, 7B, 14B
- 4K ~ 128K context length

■ Llama (2024.03)

- Meta/US
- 가장 널리 쓰이는 오픈소스 LLM
- Llama 3 License (월간 활성 사용자 7억명 이하 무료, llama 이름 포함)
- 높은 이식성: e.g. Pure C/C++ 구현 llama.cpp
 - <https://github.com/ggerganov/llama.cpp>
- 8B, 70B, 400B
- 8K context length

■ Qwen 1.5 (2024.03)

- Alibaba/US
- Tongyi Qianwen (월간사용자 1억명 이하 무료)
- 0.5B, 1.8B, 4B, 7B, 14B, 32B, 72B, 110B
- 32K context length

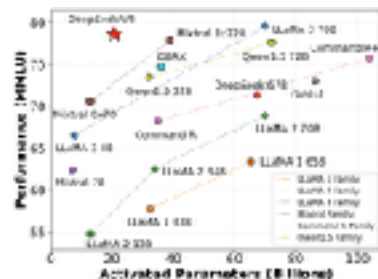
Commercial Open-source LLM (2)

- **Mixtral 8x22B v0.1 (2024.05)**

- Mistral/France
- Apache 2.0 Licence (무료)
- 141B (MoE: 39B)
- 64K context length

- **DeepSeek V2 (2024.04)**

- DeepSeek AI/US
- MIT License (무료)
- 경제성 강조
- 7B, 16B(MoE: 2.4B), 67B, 236B(MoE: 21B)
- 32K~128K context length



■ Snowflake Arctic (2024.04)

- Snowflake/US
- Apache 2.0 License (무료)
- 기업용 task 특화
- 480B (MoE: 17B)
- 4K context length

- **Falcon 2 (2024.05)**

- Technology Innovation Institute / Abu Dhabi
- Apache 2.0 License
- 11개국 언어 5000B 토큰 학습
- 11B
- 8K context length

https://llm.extractum.io/list/

All Large Language Models

Was this list helpful?

Model Size: 0 to 999B
Model VRAM: 0 to 768

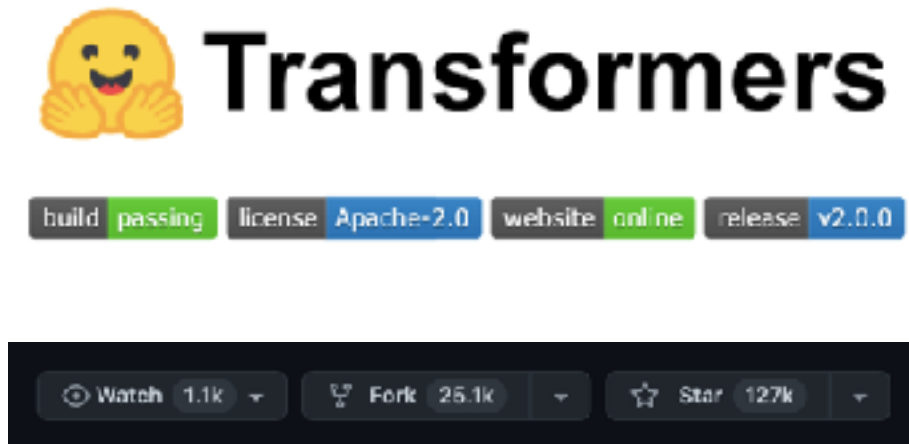
Columns Quick Filters Reset Filters & Sorting

Complete AI platform: production-ready API, latest models
Be faster | Pay less | Scale smarter
Sign up

Model Name	Maintainer	Size	Score	VRAM (GB)	Quantized	License	Context Len	Likes	Downloads
DeepSeek R1	deepseek-ai	635B	0.59	180.4	*	mit	160K	12229	735368
Meta Llama 3.8B	meta-llama	8B	0.5	16.1	*	llama3	8K	6133	5079113
Mixtral 8x7B Instruct V0.1	mistral	47B	0.42	93.6	*	proprietary	32K	4437	370846
Llama 2 7B Chat hf	meta-llama	7B	0.44	13.5	*	llama2	4K	4425	1244015
Kokoro 82M	hexgrad	82M	0.53	*	*	apache-2.0	*	1388	2010118
Llama 3.1 8B Instruct	meta-llama	8B	0.53	16.1	-	llama3.1	128K	4004	6177106
Meta Llama 3.8B Instruct	meta-llama	8B	0.48	16.1	-	llama3	8K	3973	1460980
DeepSeek V3	deepseek-ai	635B	0.66	171.8	*	*	160K	3848	807057
Mistral 7B V0.1	mistral	7B	0.37	14.4	*	proprietary	32K	3805	370973
Janus Pro 7B	deepseek-ai	7B	0.5	14.8	*	mit	16K	3388	93362
Phi 2	microsoft	3B	0.41	5.5	*	mit	2K	3337	625842
Gemma 7B 1	google	7B	0.39	17.1	gguf	gemma	8K	3158	30909
DeepSeek V3.0121	deepseek-ai	635B	0.57	184.7	*	mit	160K	2624	143319
QwQ 32B	Qwen	32B	0.72	65.8	-	apache-2.0	128K	2763	344499
Gpt2	openai-community	137M	0.26	0.5	-	mit	1K	2740	10540
Mistral 7B Instruct V0.2	mistral	7B	0.5	14.4	*	proprietary	32K	2630	331609

HuggingFace

- Machine Learning 모델 및 데이터 공유 전문 기업: LLM 기술 확산에 기여
- HuggingFace Transformer: 오픈소스 transformer 기반 관련 기술 코드 (De facto standard)
- HuggingFace Model Hub: 최신 SOTA (State-Of-The-Art) LLM 모델 체크포인트 업로드/배포
 - 3-4줄 스크립트로 최신 LLM PyTorch 모델 사용 가능
 - LLM, Image-to-Text, Video, Text-to-Audio.
- 보안상 주의: 오픈소스 플랫폼의 특성으로 기성 플랫폼에 비해 불안정성, 보안취약점 보고



<https://github.com/huggingface/transformers>

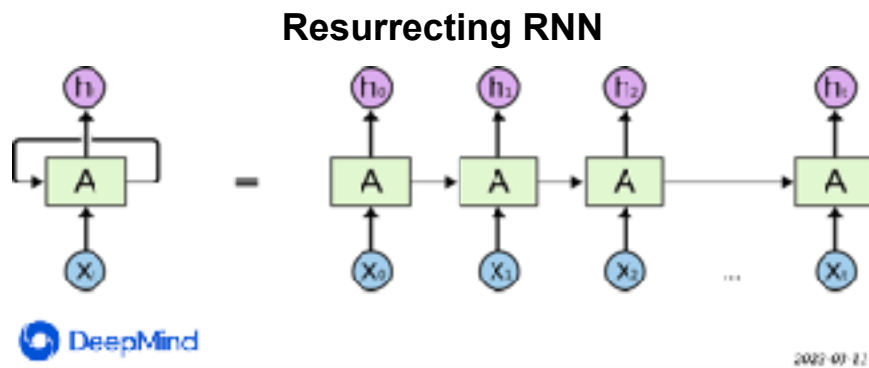


<https://huggingface.co/models>

Concluding Remarks

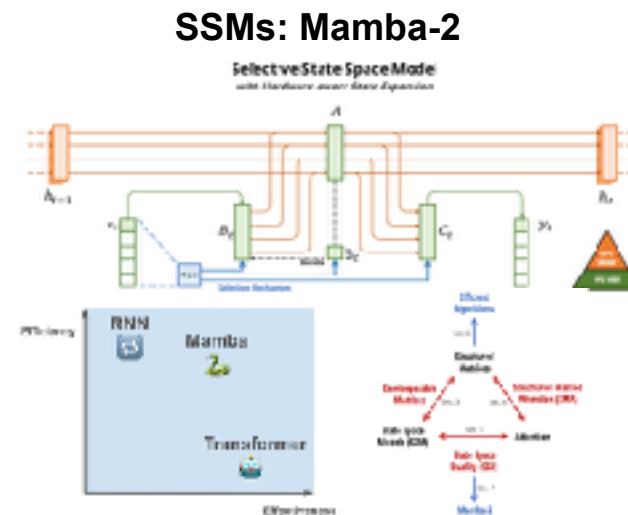


<https://youtu.be/1EAsn71O9yA?si=Dy1Lkid6AfAhLI4g>

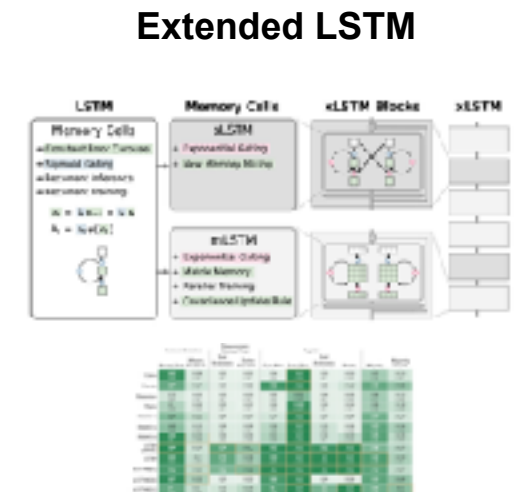


Resurrecting Recurrent Neural Networks for Long Sequences

<https://arxiv.org/abs/2303.06349>



<https://arxiv.org/abs/2405.21060>



<https://arxiv.org/pdf/2405.04517v1>